

Metagenomics Comparison

Lorenzo Berlese¹ and Matteo Meneghin¹

¹Università degli Studi di Padova

July 9, 2025

Abstract

This report is focused on the applicability of non-exact clustering algorithms such as CLARANS to find a good clustering of metagenomic samples. The samples were obtained from the Global Ocean Sampling Expedition (GOS) dataset and consist of 64 distinct files. To address the computational limitations of our implementation of the Bray-Curtis distance for clustering, we used Simka to process the data and compute the distance matrix. From this matrix, we derived a secondary matrix that served as a counter, tracking how many times each distance value was accessed. The results have shown that it is possible to make a clustering that does not calculate all distances keeping a good quality. This work sets the basis for future analysis on dendrogram creation starting from these clusterings, with the idea of testing the similarity between them and dendrograms created starting from an all-against-all distance matrix.

1 Introduction

What is metagenomics Metagenomics is the application of modern genomic techniques to the study of communities of microbial organisms directly in their natural environment (Chen and Pachter [2005]). This approach bypasses the need to isolate and cultivate individual species in the laboratory, which is particularly beneficial since many bacteria and viruses cannot survive in isolation.

In principle, it is possible to access 100% of the genetic resources of an environment.

The applications are many:

- Plant-microbe interaction for improving crop yield
- Monitoring and optimizing wastewater treatment
- Bioprospecting: hunting for novel enzymes and compounds
- Function and ecology of the oceans
- Development of functional feed
- Monitoring pathogens
- Monitoring pollution
- Disease surveillance and treatment

The metagenomics workflow (figure 1), described in Thomas et al. [2012], has several steps.

Sampling and Processing: This is the initial and most critical step. The primary goal is to extract DNA that is representative of all cells present in the sample and to obtain sufficient amounts of high-quality nucleic acids for subsequent analysis.

Specific protocols are often required for different sample types. Special considerations include minimizing host DNA contamination when the target community is associated with a host, which can be achieved through fractionation or selective lysis. Physical fractionation can also enrich specific target parts of the community, such as viruses.

Maximizing DNA yield and avoiding co-extraction of enzymatic inhibitors, like humic acids in soil, are important, and the method of lysis can introduce biases.

For samples yielding very small amounts of DNA, amplification methods may be necessary to increase DNA yields from femtograms to micrograms. However, these methods carry potential risks that can significantly affect downstream analysis.

Sequencing Technology: The metagenomic shotgun sequencing uses two types of sequencing.

Sanger sequencing, while considered the gold standard for its low error rate, long read length (>700 bp), and large insert sizes, is labor-intensive, prone to bias against genes toxic to cloning hosts, and very costly (~USD 400,000 per gigabase). It remains applicable for generating near-complete genomes in low-diversity environments.

NGS technologies, such as the 454/Roche and Illumina/Solexa systems, have been extensively applied due to their much lower cost (e.g., ~USD 20,000 per gigabase for 454/Roche; ~USD 50 per gigabase for Illumina/Solexa). These technologies have different read lengths and error characteristics; for instance, 454/Roche can have issues with homopolymers leading to insertion/deletion errors, while Illumina/Solexa might show higher error rates at the tail ends of reads. NGS platforms also offer multiplexing, allowing analysis of multiple samples in a single run. The read length of the technology chosen impacts the ability to perform functional annotation and assembly.

A different approach is the marker gene approach, based on the sequencing of a specific gene region. It allows us to describe the microbial composition based on the taxonomic groups present in the sample. It is frequently used to analyze the biodiversity of microbial ecosystems.

Assembly: This step is performed when the research aims to recover the genomes of uncultured organisms or obtain full-length coding sequences. It involves assembling short read fragments into longer genomic contigs. A key challenge is that most existing assembly programs were designed for single, clonal genomes, making their utility for complex metagenomic mixtures require careful evaluation. The assumption of clonality can lead to the suppression of contig formation for heterogeneous taxa. Two main strategies exist:

- Reference-based assembly is effective when closely related reference genomes are available, offering speed and memory efficiency. However, it can result in fragmented assemblies or un-covered divergent regions if the sample's true genome differs significantly from the reference.
- De novo assembly typically demands greater computational resources. While traditional de Bruijn graph assemblers have high computational requirements, newer "metagenomic assemblers" explicitly address the non-clonality of natural populations by identifying subgraphs or partitioning sequence mixes into "species bins".

Assembly is important because longer sequence information generally improves the accuracy of annotation and benefits the binning and classification of DNA fragments. It also increases the quality and confidence of genetic information by merging multiple reads covering the same segment.

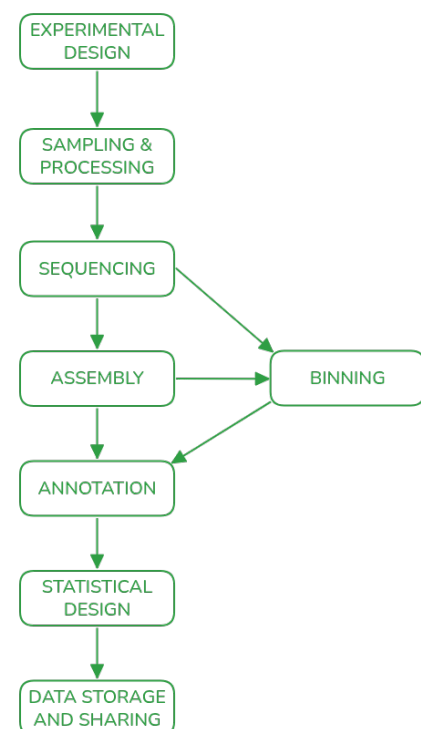


Figure 1: Metagenomics workflow

Binning: In this step DNA sequences are grouped into bins, potentially representing individual or related genomes. Binning algorithms use compositional features or similarity to known genes, and their reliability can depend on read length and available reference data. This process involves sorting DNA sequences into groups that likely represent individual genomes or genomes from closely related organisms. Binning algorithms use two main types of information:

- Compositional binning leverages the conserved nucleotide composition of genomes, such as GC content or k -mer abundance distribution. This method is generally not reliable for short reads due to insufficient information but can be enhanced with training datasets.
- Similarity-based binning classifies unknown DNA fragments by comparing encoded genes to known genes in reference databases. This requires the availability of reference data, and distant relationships may only allow high-level taxonomic assignments.

Some algorithms combine both compositional and similarity information. Binning can be performed post-assembly to generate partial genomes of unknown organisms, or pre-assembly to reduce the complexity of the assembly effort.

Annotation: This step involves two primary phases: identifying features of interest (genes) and then assigning putative gene functions and taxonomic neighbors. For reconstructed genomes, established genome annotation pipelines are used, but for unassembled reads or short contigs from entire communities, specialized metagenomic analysis tools are necessary. Functional annotation relies on classifying sequences to known functions or taxonomic units through homology searches against available "annotated" data.

Experimental Design and Statistical Analysis: Plan experiments with proper replication and apply statistical tools for data analysis ensure that observed patterns are statistically significant and distinguishes between biological and technical variation. Experimental design should be driven by the research question, including taking suitable reference samples and processing them consistently. It's crucial to differentiate between true biological variation and technical variation. Microbial systems are often dynamic, so temporal aspects of sampling should be considered. Replication is paramount: multiple samples from a habitat, processed separately, are needed to understand biological variation. Simply splitting a single sample only reveals technical variation, and pooling samples loses all variability information. Small-scale preliminary experiments can help estimate the magnitude of variation.

Data Storage and Sharing: Manage and share metagenomic data using centralized services and standard formats is crucial for comparing individual projects and utilizing rich information from other metagenomic projects. Data sharing is a long-standing tradition in genome research, but for metagenomic data, it requires new levels of organization and collaboration, including metadata and centralized services.

The problem Our project does not follow this workflow but instead focuses on comparing metagenomic samples. This comparison has several applications in precision medicine. For example, samples can be grouped based on diet or environmental conditions. It is also useful for routine monitoring of antibiotic resistance in wastewater and affected aquatic environments, where samples can be grouped by location or time.

Given a set of metagenomics sample datasets, their all-against-all-comparison is a necessary step to derive a hierarchical clustering among the samples. However, it is also a very expensive step from a computational point of view. The key steps of our work are the computation of the distance matrix between all samples using Simka and the implementation of the fastCLARANS algorithm, a data mining technique designed to avoid computing all pairwise distances and to evaluate how many distance computations can potentially be skipped.

We investigated these steps using a real metagenomic dataset and explored different parameter settings for the fastCLARANS algorithm to assess the trade-off between accuracy and computational time.

2 Methods

2.1 Clustering algorithms

PAM The PAM algorithm is very similar to k -means, mostly because both are partitional algorithms, in other words, both break the dataset into groups (clusters), and both work by trying to minimize a type of error (distance), but PAM solves k -medoids. Medoids are entities of the dataset that represent the group in which it is inserted, while k -means works with centroids, that are artificially created entity that represent its cluster.

The PAM algorithm partitions the dataset of n objects into k clusters, where both the dataset and the number k is an input of the algorithm. This algorithm works with a matrix of dissimilarity, and its goal is to minimize the overall dissimilarity between the representants of each cluster and its members.

The distance function k -medoids minimizes is the Total Deviation (TD) of each element from its medoid, that is:

$$TD := \sum_{i=1}^k \sum_{x_j \in C_i} d(x_j, m_i) \quad (1)$$

where C_i is a cluster, x_j one element of the dataset, m_i is the medoid of cluster C_i

The medoid of a set C is the element of the set that minimizes the sum of dissimilarities to all the other elements of the set (Schubert and Rousseeuw [2019]):

$$medoid(C) := \arg \min_{x_i \in C} \sum_{x_j \in C} d(x_i, x_j) \quad (2)$$

Algorithm

Input:

O : set of objects

S : set of selected objects, medoids

$U = O - S$: set of unselected objects

Define:

- $d_{nearest}(x_j)$: dissimilarity between x_j and the closest object in S , $x_j \in S$ if and only if $D_p = 0$
- $d_{second}(x_j)$: the dissimilarity between x_j and the second closest object in S , $d_{nearest}(x_j) \leq d_{second}(x_j)$

The idea behind PAM is to find an initial set of k medoids through the BUILD phase, where one after the other a medoid is added. The rule for this adding is to find the element in U that lowers the most the $d_{nearest}(x_o)$ values considering all elements $x_o \in U$.

After that, the SWAP phase is thought to lower the value of TD by repeatedly swapping a non-medoid with a medoid. The choice for the pair (m_i, x_j) to swap is done considering all the pairs possible and measuring the sum of the changes in TD for a given pair, looping on all the other elements x_o .

The formula for this change is the following (Schubert and Rousseeuw [2019]):

$$\Delta TD(x_o, m_i, x_j) = \begin{cases} \min\{d(x_o, x_j), d_{second}(o)\} - d_{nearest}(o) & \text{if } i = nearest(o) \\ \min\{d(x_o, x_j) - d_{nearest}(o), 0\} & \text{otherwise} \end{cases}$$

```

1 select the element of  $U$  that minimizes the distance from all other elements
2 for  $medoid \in (1, \dots, k - 1)$  do
3    $\Delta TD_{max} = 0$ 
4   foreach  $x_j \in U$  do
5     select  $x_j \in U$ 
6      $\Delta TD_j = 0$ 
7     foreach  $x_i \in U - \{x_j\}$  do
8        $C_{jo} = \max\{d_{nearest}(x_o) - d(x_j, x_o), 0\}$ 
9        $\Delta TD_j = \Delta TD_j + C_{jo}$ 
10    end
11    if  $\Delta TD_j > \Delta TD_{max}$  then
12       $\Delta TD_{max} = \Delta TD_j$ 
13       $entering = x_j$ 
14    end
15  end
16   $S = S \cup \{entering\}$ 
17   $U = U - \{entering\}$ 
18 end
19 repeat
20    $(\Delta TD^*, m^*, x^*) \leftarrow (0, \text{null}, \text{null})$  foreach  $(m_i, x_j) \in S \times U$  do
21      $\Delta TD = 0$ 
22     foreach  $x_o \in U - \{x_j\}$  do
23        $\Delta TD(x_o, m_i, x_j) =$ 
24         
$$\begin{cases} \min\{d(x_o, x_j), d_{second}(o)\} - d_{nearest}(o) & \text{if } i = nearest(o) \\ \min\{d(x_o, x_j) - d_{nearest}(o), 0\} & \text{otherwise} \end{cases}$$

25        $\Delta TD = \Delta TD + \Delta TD(x_o, m_i, x_j)$ 
26     end
27     if  $\Delta TD < \Delta TD^*$  then
28        $(\Delta TD^*, m^*, x^*) \leftarrow (\Delta TD, m_i, x_j)$ 
29     end
30   end
31   swap  $m_i$  and  $x_j$ 
32   update  $d_{nearest}$  and  $d_{second}$  for every  $x_o \in U$ 
33 until  $\Delta TD^*(x_o, m_i, x_j) > 0$ ;

```

CLARANS A way to reduce the number of distances calculated to create a clustering could be the use of the CLARANS algorithm, name that stands for Clustering Large Applications based on RANdomized search.

The CLARANS algorithm is different from PAM and CLARA (PAM repeatedly applied to a sub-sample of the dataset): it is not performed a check for every pair (medoid, non-medoid) to find a possible diminishing of the loss function (sum of distances from the nearest medoid), instead it is thought an underlying graph abstraction of the problem, $G_{n,k}$, where each node is a collection of k points, representing the medoids and every arch is the swap of a medoid with a non-medoid.

The CLARANS algorithm searches in this highly connected graph for some neighbours that bring a lowering of the loss function, and it does so until it finds a local minimum. Starting from k random medoids, at every step, the number of neighbours checked is not the entire set of nodes at distance 1 from the current one, instead there is a parameter, *maxneighbor*, that manages the level of time spent on a node.

Moreover, after finding a certain amount (*numlocal*) of local minima, the one that leads to the lowest value of the loss function is returned.

It is important to note that, although this algorithm does not guarantee the discovery of exact local minima, and instead considers a node to be a local minimum if none of the explored neighbours improve the loss function, the results presented in Ng and Han [2002] show that CLARANS still finds solutions comparable in quality to those of PAM, which instead performs an exhaustive search of all neighboring medoid sets.

Adding to this, it is clear that being the graph highly connected, moving to the real local minimum does not suffer highly from "taking the wrong direction", because there are multiple routes that bring to the same pseudo-optimal node.

The pseudocode for CLARANS (Ng and Han [2002]) is the following:

- 1 Input parameters *numlocal* and *maxneighbor*. Initialize *i* to 1, and *mincost* to a large number.
- 2 Set *current* to an arbitrary node in $G_{n,k}$.
- 3 Set *j* to 1.
- 4 Consider a random neighbor *S* of *current*, and based on the formula for *TD* or ΔTD , calculate the cost differential of the two nodes.
- 5 If *S* has a lower cost, set *current* to *S*, and go to Step 3.
- 6 Otherwise, increment *j* by 1. If $j \leq \text{maxneighbor}$, go to Step 4.
- 7 Otherwise, when $j > \text{maxneighbor}$, compare the cost of *current* with *mincost*. If the former is less than *mincost*, set *mincost* to the cost of *current* and set *bestnode* to *current*.
- 8 Increment *i* by 1. If $i > \text{numlocal}$, output *bestnode* and halt. Otherwise, go to Step 2.

In line 5, the cost of the near node could be calculated taking the sum of the distances of every point from the nearest medoid (so recalculating it every time) or it can be kept track of just the *change* that a different set of medoids brings. If this change is < 0 , the swap is convenient.

fastCLARANS In addition to CLARANS, it has been implemented an improvement that permits to check more neighbours (*k*) in the same iteration and select the best one for swapping. According to Schubert and Rousseeuw [2019], this novel feature comes at no additional cost in terms of speed and space, meaning they are needed less swaps to reach a local optimum.

The above mentioned considerations are implemented in the original paper and named "fastPAM1", because relative to the SWAP phase of PAM algorithm. fastPAM1 can indeed be adopted also in CLARA and CLARANS.

The rationale of fastPAM1 is that, instead of selecting one medoid m_i and one non-medoid x_j to see if their swap brings a lowering of the objective function, only x_j is selected at random. Then an array ΔTD of size k (= number of medoids) is kept, containing the *change* ΔTD_i in the objective function if the swap was performed with that particular medoid.

First, ΔTD is lowered in each of its cells by a quantity equal to the distance of the selected non-medoid x_j from its nearest center (done in $O(k)$ time). Then it is calculated the change in objective function given by the contribution of every point x_o other than the medoids and the selected non-medoid x_j ($O(k)$ every time $d_{oj} < d_n$).

The pseudocode for fastCLARANS is the following:

```

1 repeat
2    $(\Delta TD^*, m^*, x^*) \leftarrow (0, \text{null}, \text{null})$ ;           // Empty best candidate storage
3   foreach  $x_j \notin \{m_1, \dots, m_k\}$  do
4      $d_j \leftarrow d_{\text{nearest}}(x_j)$ ;           // Distance to current medoid
5      $\Delta TD \leftarrow (-d_j, -d_j, \dots, -d_j)$ ;           // Change if making  $j$  a medoid
6     foreach  $x_o \neq x_j$  do
7        $d_{oj} \leftarrow d(x_o, x_j)$ ;           // Distance to new medoid
8        $(n, d_n, d_s) \leftarrow (\text{nearest}(o), d_{\text{nearest}}(o), d_{\text{second}}(o))$ ;           // Cached values
9        $\Delta TD_n \leftarrow \Delta TD_n + \min\{d_{oj}, d_s\} - d_n$ ;           // Loss change
10      if  $d_{oj} < d_n$  then // Reassignment check
11        foreach  $m_i \in \{m_1, \dots, m_k\} \setminus m_n$  do
12           $\Delta TD_i \leftarrow \Delta TD_i + d_{oj} - d_n$ ;           // Update loss change
13        end
14      end
15    end
16     $i \leftarrow \arg \min \Delta TD_i$ ;           // Choose best medoid  $i$ 
17    if  $\Delta TD_i < \Delta TD^*$  then
18       $(\Delta TD^*, m^*, x^*) \leftarrow (\Delta TD_i, m_i, x_j)$ ;           // Store
19    end
20  end
21  if  $\Delta TD^* \geq 0$  then
22    break
23  end
24  // Swap roles of medoid  $m^*$  and non-medoid  $x^*$ 
25   $TD \leftarrow TD + \Delta TD^*$ 
26 until;
27 return  $TD, M, C$ 

```

This procedure fits into the lines 4 and 5 of the CLARANS pseudocode, with some adjustments. In particular, at line 6 of fastPAM it is not performed a cycle along all the non medoids, but it is used the logic of the "neighbors move", meaning that the number of swaps tried is not the maximum possible, but only the quantity that leads to a local minimum.

2.2 Simka

Current methods for comparing microbial communities face significant limitations (Benoit et al. [2015]).

Reference-based methods, which rely on taxonomical or functional assignation, only use a small subset of sequences that can be linked to known organisms. This means they may not capture the entire microbial diversity of a sample and are often impractical for whole metagenomic sets due to computational cost and leaving out a large fraction of reads.

De novo methods, which compare whole sets of sequences, often struggle with scalability for ambitious metagenomic projects or do not provide precise and exhaustive results. For example, early approaches using Blast couldn't scale to large studies with dozens or hundreds of read sets.

Mash, which performs a sub-sampling of the k -mers using the MinHash approach, while being highly computationally efficient due to k -mer sub-sampling, comes at the expense of result quality and precision. Its output distances do not account for relative abundance information and are not computed exactly. This is a significant drawback because differences in relative abundances are often at the heart of interesting biological insights in comparative genomics studies.

Simka compares N metagenomic datasets based on their k -mer counts to compute a collection of ecological distances. The core idea is to replace species counts, traditionally used in ecology, with k -mer counts. The computation of the $N \times N$ distance matrix D , where $D_{i,j}$ represents an ecological distance between datasets S_i and S_j , is theoretically broken down into two main steps:

1. **Multiset k -mer Counting (MKC):** The objective of this step is to represent each dataset as a set of discriminant features, specifically, k -mer counts. Traditionally, this would involve creating a large k -mer count matrix of size $W \times N$ (where W is the number of distinct k -mers), but storing this matrix is computationally prohibitive for large datasets.

Simka overcomes this by proposing an alternative computation scheme where each dataset independently extracts, canonicalizes, and partitions its k -mers, which are sorted and counted locally (in P parts), with strong parallelism and compressed disk storage. P processes can run independently, generating P streams of abundance vectors in parallel, a crucial point is that these abundance vectors are not stored; they are immediately used as input streams for the next step: k -mer abundance filter.

2. **Distance Computation:** A matrix ($M_{\cap}$) is initialized to record the final crossed terms for each pair of datasets. P independent processes run in parallel and each process iterates over its stream of abundance vectors. For each k -mer's abundance vector, it loops through every possible pair of datasets (S_i, S_j). If a k -mer is shared between two datasets (i.e., has a positive abundance in both), the partial crossed term matrix ($M_{\cap\text{part}}$) is updated. When all streams are done, the algorithm reads each written $M_{\cap\text{part}}$ and accumulates it to $M_{\cap}$.

Simka can compute a collection of ecological distances for all pairs of datasets. We use Simka to compute the Bray–Curtis distance, defined as:

$$\text{BrayCurtis}_{Ab}(S_i, S_j) = 1 - 2 \frac{\sum_{w \in S_i \cap S_j} \min(N_{S_i}(w), N_{S_j}(w))}{\sum_{w \in S_i} N_{S_i}(w) + \sum_{w \in S_j} N_{S_j}(w)} \quad (3)$$

where w is a k -mer and $N_{S_i}(w)$ is the abundance of w in the dataset S_i .

2.3 Evaluation metrics

Evaluating how well the clustering has performed can be approached as a classification problem. In a typical binary classification setting, predicted values are compared with the true (actual) values, and each prediction can be categorized as either Positive or Negative.

When evaluating classification results, there are four possible outcomes:

- TP (True Positive): The predicted value is Positive, and the actual value is also Positive.
- FP (False Positive): The predicted value is Positive, but the actual value is Negative.
- FN (False Negative): The predicted value is Negative, but the actual value is Positive.
- TN (True Negative): The predicted value is Negative, and the actual value is also Negative.

Commonly used metrics are:

- Precision, which measures how many of the predicted Positive values are actually Positive: $TP/(TP+FP)$
- Recall, which measures how many of the actual Positive values are correctly predicted: $TP/(TP+FN)$
- F1-score, the harmonic mean of precision and recall: $2 \times (\text{precision} \times \text{recall})/(\text{precision} + \text{recall})$

But in this project we are dealing with a multi-class classification task, we need to categorize each sample into 1 of N different classes (Chase [2019b], Chase [2019a]). We need to combine the per-class scores into a single number, the classifier's overall score.

Define:

- N : total number of classes
- n : total number of actual values

- x_α : number of actual values for the class α
- P_α : precision for the class α
- R_α : precision for the class α
- $F1_\alpha$: F1-score for the class α

Macro-averaged scores Each score is computed as a simple arithmetic mean of our per-class scores.

$$P_{\text{macro-averaging}} = \frac{P_A + P_B + \cdots + P_N}{N} \quad (4)$$

$$R_{\text{macro-averaging}} = \frac{R_A + R_B + \cdots + R_N}{N} \quad (5)$$

$$F1_{\text{macro-averaging}} = \frac{F1_A + F1_B + \cdots + F1_N}{N} \quad (6)$$

Weighted-averaged scores The scores of each class are weighted by the number of samples from that class, differently from macro-averaged where each class has same weight.

$$P_{\text{weighted-averaging}} = \frac{P_A x_A + P_B x_B + \cdots + P_N x_N}{n} \quad (7)$$

$$R_{\text{weighted-averaging}} = \frac{R_A x_A + R_B x_B + \cdots + R_N x_N}{n} \quad (8)$$

$$F1_{\text{weighted-averaging}} = \frac{F1_A x_A + F1_B x_B + \cdots + F1_N x_N}{n} \quad (9)$$

Micro-averaged scores Gives equal weight to each instance. We first compute micro-averaged precision and micro-averaged recall over all the samples, and then combine the two.

$$P_{\text{micro-averaging}} = \frac{TP_A + TP_B + \cdots + TP_N}{TP_A + FP_A + TP_B + FP_B + \cdots + TP_A + FP_N} \quad (10)$$

$$R_{\text{micro-averaging}} = \frac{TP_A + TP_B + \cdots + TP_N}{TP_A + FN_A + TP_B + FN_B + \cdots + TP_A + FN_N} \quad (11)$$

We can observe that $FP_A + FP_B + \cdots + FP_N = FN_A + FN_B + \cdots + FN_N$, because are all the values falsely classified. So

$$R_{\text{micro-averaging}} = P_{\text{micro-averaging}} = F1_{\text{micro-averaging}} \quad (12)$$

2.4 Dataset

The dataset used for the report consists of one project of the GOS (Global Ocean Sampling) dataset, which contains reads sequenced with Sanger (prevalently) and pyrosequencing.

The samples we have chosen are in the tables 1 and 2, listed with their geographical label, which is where the sample had been taken, and the number of reads present in each sample.

The dataset repository is available at this URL: <https://datacommons.cyverse.org/browse/iplant/home/shared/imicrobe/projects/26/samples>

Sample ID	Geographical Label	Num. Reads
GOS02	North Amer. East Coast	121590
GOS03	North Amer. East Coast	61605
GOS04	North Amer. East Coast	52959
GOS05	North Amer. East Coast	61131
GOS06	North Amer. East Coast	59679
GOS07	North Amer. East Coast	50980
GOS08	North Amer. East Coast	129655
GOS09	North Amer. East Coast	79303
GOS10	North Amer. East Coast	78304
GOS11	North Amer. East Coast	124435
GOS12	North Amer. East Coast	722
GOS13	North Amer. East Coast	138033
GOS14	North Amer. East Coast	128885
GOS15	Caribbean Sea	127362
GOS16	Caribbean Sea	127122
GOS17	Caribbean Sea	257581
GOS18	Caribbean Sea	142743
GOS19	Caribbean Sea	135325
GOS20	Panama Canal	296355
GOS21	Eastern Tropical Pacific	131798
GOS22	Eastern Tropical Pacific	121662
GOS23	Eastern Tropical Pacific	133051
GOS24	Eastern Tropical Pacific	120671
GOS25	Galapagos Islands	1284
GOS26	Galapagos Islands	222080
GOS27	Galapagos Islands	189052
GOS28	Galapagos Islands	131529
GOS29	Galapagos Islands	358028
GOS31	Galapagos Islands	892
GOS32	Galapagos Islands	692255
GOS33	Galapagos Islands	134347

Table 1: Sample metadata: Part 1 of 2

Sample ID	Geographical Label	Num. Reads
GOS34	Galapagos Islands	140814
GOS35	Galapagos Islands	77538
GOS36	Eastern Tropical Pacific	65670
GOS46	Southern Tropical Pacific	66023
GOS47	Polynesia Arch.	90515
GOS48	Polynesia Arch.	92501
GOS49	Polynesia Arch.	715
GOS50	Polynesia Arch.	128982
GOS51	North Amer. East Coast	1334
GOS52	Polynesia Arch.	47691
GOS53	Indian Ocean	51787
GOS54	Indian Ocean	49595
GOS55	Indian Ocean	59812
GOS56	Indian Ocean	99288
GOS57	Indian Ocean	49597
GOS59	Indian Ocean	59079
GOS60	Indian Ocean	99781
GOS61	Indian Ocean	52118
GOS62	Indian Ocean	718
GOS63	Indian Ocean	9741
GOS64	Indian Ocean	61020
GOS65	Indian Ocean	60932
GOS66	Indian Ocean	346952
GOS67	Indian Ocean	50609
GOS68	Indian Ocean	60987
GOS69	Indian Ocean	46052
GOS70	Indian Ocean	110720
GOS71	Indian Ocean	10720
GOS72	Indian Ocean	50095
GOS73	Indian Ocean	107966
GOS74	Indian Ocean	96463
GOS75	Indian Ocean	99271
GOS77	Indian Ocean	1156475

Table 2: Sample metadata: Part 2 of 2

3 Our work

Our pipeline Starting from the downloaded dataset, we run Simka on the BLADE cluster to create a distance matrix, computed using the Bray-Curtis distance on k -mers with $k = 21$. The distance matrix is then used in our CLARANS implementation, where each sample is assigned a cluster label. For each cluster, we determine the majority label, which we treat as the predicted label. Then we use standard metrics to assess clustering performance.

Use of Simka We use Simka with the default parameters via the DEI cluster, obtaining the distance matrix for the 64 samples in 29 minutes and 39 seconds, with a memory usage of 29.4 GB.

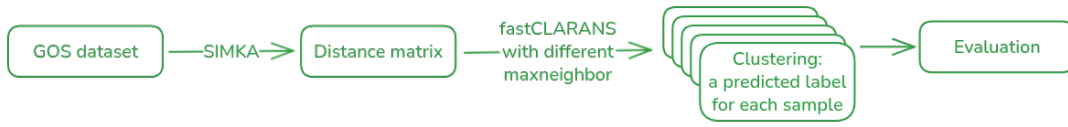


Figure 2: Our pipeline

CLARANS implementation We develop in Python the CLARANS algorithm, in which we use the Bray-Curtis distance. The algorithm has different input parameters:

- `samples`: the list of samples, each sample is represented by a dictionary with the following keys:
 - `id` (str): Identifier for the sample, retrieved from the filename in the format `GOSXX`.
 - `center` (int): Index of nearest medoid for the sample. Initialized to -1. Its value ranges from 0 to $k - 1$ because it is an index with respect to the set of medoids.
 - `second` (int): Index of second-nearest medoid for the sample. Initialized to -1. Its value ranges from 0 to $k - 1$ because it is an index with respect to the set of medoids.
 - `ambiental_label` (str): Additional label for environmental context, retrieved from a file with the list of true labels.
- `k`: the number of clusters desired. In our case 8 because the dataset has 8 distinct labels.
- `numlocal`: the number of local minima to search. We set it to 3 following the results of Ng and Han [2002] that shows how using a higher value for this parameter brings a low increase in clustering quality at the cost of higher runtime.
- `maxneighbor`: the number of neighbours of a node in the graph after which a local minimum is assessed, meaning that no further exploration is performed from this point. We experiment with different values, starting from $\text{num_clusters} \cdot (n - \text{num_clusters}) = 448$, which corresponds to all possible configurations obtained by swapping one medoid with one non-medoid, simulating the PAM algorithm. Then we try these other values: 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 20, 30, 40, 50, 60, 70, 80, 90, 100, 200, 300, 400.
- `distance_matrix`: the 64×64 matrix of Bray-Curtis distances computed using Simka converted to NumPy Array

Moreover, the distances between samples have not been recalculated at each call of fastCLARANS, but every time a local matrix has been updated with a flag whenever a not-already-used distance has been accessed.

For the fact that fastCLARANS is a non-exact algorithm, for each value considered of *maxneighbors*, we run fastCLARANS 50 times, so to obtain a result not due to randomness.

Evaluation For evaluation, we need to define the correct label for each cluster. We choose this label based on frequency, specifically, the most frequent one.

The `findmajority_label` function takes as input a list of dictionaries, where each dictionary represents a sample and contains:

- `id`: the sample identifier;
- `center`: the cluster ID assigned to the sample;
- `second`: an optional secondary cluster label;
- `ambiental_label`: the environmental category of the sample, its true label.

The function groups samples by their cluster label and uses a majority vote (based on frequency) to find the most common `ambiental_label` in each cluster.

Returns a list of dictionaries, each containing:

- `cl_id`: the cluster ID;
- `majority_label`: the most common environmental label in that cluster.

The function `compute_evaluation_metrics` takes as input a list of dictionaries in the same format as `find_majority_label`, and returns a dictionary. This dictionary contains, for each label, a sub-dictionary with the values of TP, FN, FP, and TN. It also includes three additional dictionaries containing the recall, precision, and F1-score values computed as macro-average, weighted-average, and micro-average, as introduced in Section 2.3.

Alternative solutions tried before For parsing the dataset and creating a data structure to contain the k -mers and apply the Bray-Curtis distance formula on them, we first tried to store every k -mer found through parsing in a Counter, that is a Python dictionary. This solution was shown to be affordable in our research in terms of time spent to prepare the various samples, with a time of minutes for each sample, but it has had the drawback of calculating the Bray-Curtis distance in too much time (order of magnitude of tens of minutes for a single distance).

Another strategy that has been implemented and tested is the use of a `csr_matrix`, a memory-efficient data structure from `scipy.sparse` for storing sparse matrices. We create a one-row matrix where the column indices correspond to the unique base-4 encoding of each valid k -mer, and the values at those positions represent their frequencies in the sequence. This approach enables efficient vectorization of distance metrics such as the Bray-Curtis distance. However, it turned out to be too slow during the data parsing phase of the algorithm (order of magnitude of hours for a single sample).

Due to the limited time available for the research, the Simka solution allowed for a fast procedure both for parsing the FASTA files and for calculating the distances between them. This had the side effect of enabling quicker debugging, allowing us to promptly identify any bugs in the code, rather than having to wait for a long BLADE computation.

4 Results

The results show that the increase of the value of *maxneighbor* is not effective in terms of the quality of clustering: either the raw value of the mean and the value of the uncertainty for F1 score, precision and recall do not change (Table 4 and Table 5).

Looking at the calculated number of distances (in Table 3 and Figure 4), it is possible to see that starting from the value of *maxneighbour* = 20 and taking higher values, the number of distances is constant and almost equal to the maximum number of distances, which is $n \cdot n = 64 \cdot 64 = 4096$, where n is the number of metagenomic samples. This means that for these values just the 2% of distance computations have been avoided (for the best case in this range, that is, *maxneighbour* = 20).

The real change is seen with the values between 1 and 10, because the number of distances needed to reach a clustering ranges from 1827.440 ± 33.600 to 3728.210 ± 39.349 , leading to the avoidance of a number of distances between 9% (for *maxneighbour* = 10) and 55% (for *maxneighbour* = 1).

Concerning run-times, it is clear from Table 3 that a higher value of *maxneighbour* leads to longer computations. In particular, the behaviour appears to be linear (Figure 3).

5 Conclusions

The experiment carried out shows that the use of fastCLARANS for clustering metagenomic samples leads to results whose quality does not change in function of the parameter *maxneighbor*. The runtime

of such algorithm is instead affected, because the parameter chosen for the research hypothesis tunes the level of thoroughness in searching a local minimum, with higher values being more thorough than lower ones.

The quality of clustering, despite the fact that is very similar for all results, also show a level of precision, recall, and F1 score, in each three versions considered, that is not considerably elevate. This is probably due to the fact that the distance matrix computed by Simka at the first step of our pipeline contained values high for the majority of them, not showing clear similar samples (and evident clusters). The use of the geographic provenience of the sample appears to be not restrictive in terms of habitat present in that site.

For what concerns the number of distances calculated for the running of the algorithm, the levels near 4096 obtained in correspondence of values of *maxneighbor* higher than 50 could be explained by the existence of a "switch" value that permits the algorithm to overcome suboptimal local minima and explore bigger parts of the search space (of dimension $\binom{64}{8} = 4426165368$).

For this reason, it is probable that the exactly equal values found for F1 score, precision and recall for the higher values of *maxneighbor* are due to the finding of the same global minimum.

A possible prosecution of this research is the creation of a dendrogram from the clusters computed by fastCLARANS, and compare it with a dendrogram created from the full distance matrix. Such comparison could be performed through the use of the PHYLIP tool after a proper translation of the dendrograms in the Newick format.

References

Gaetan Benoit, Pierre Peterlongo, Dominique Lavenier, and Claire Lemaitre. Simka: fast kmer-based method for estimating the similarity between numerous metagenomic datasets. 07 2015.

Will Chase. Multi-class metrics made simple, part ii: The f1 score, 2019a. URL <https://medium.com/data-science/multi-class-metrics-made-simple-part-ii-the-f1-score-ebe8b2c2ca1>. Accessed: 2025-07-08.

Will Chase. Multi-class metrics made simple, part i: Precision and recall, 2019b. URL <https://medium.com/data-science/multi-class-metrics-made-simple-part-i-precision-and-recall-9250280bddc2>. Accessed: 2025-07-08.

K Chen and L Pachter. Bioinformatics for whole-genome shotgun sequencing of microbial communities. *PLoS Computational Biology*, 1(2):e24, 2005. doi: 10.1371/journal.pcbi.0010024. URL <https://doi.org/10.1371/journal.pcbi.0010024>.

R.T. Ng and Jiawei Han. Clarans: a method for clustering objects for spatial data mining. *IEEE Transactions on Knowledge and Data Engineering*, 14(5):1003–1016, 2002. doi: 10.1109/TKDE.2002.1033770.

Erich Schubert and Peter J. Rousseeuw. *Faster k-Medoids Clustering: Improving the PAM, CLARA, and CLARANS Algorithms*, page 171–187. Springer International Publishing, 2019. ISBN 9783030320478. doi: 10.1007/978-3-030-32047-8_16. URL http://dx.doi.org/10.1007/978-3-030-32047-8_16.

Torsten Thomas, Jack Gilbert, and Folker Meyer. Metagenomics - a guide from sampling to data analysis. *Microbial Informatics and Experimentation*, 2(1):3, 2012. doi: 10.1186/2042-5783-2-3. URL <https://doi.org/10.1186/2042-5783-2-3>.

MaxNeighbors	#Distances	Runtime (s)
1	1827.440 $\pm$ 33.600	0.011 $\pm$ 0.001
2	2292.500 $\pm$ 46.785	0.017 $\pm$ 0.002
3	2637.280 $\pm$ 52.293	0.018 $\pm$ 0.002
4	2987.280 $\pm$ 52.077	0.023 $\pm$ 0.002
5	3154.560 $\pm$ 55.112	0.024 $\pm$ 0.002
6	3307.730 $\pm$ 51.111	0.031 $\pm$ 0.003
7	3490.800 $\pm$ 45.912	0.039 $\pm$ 0.004
8	3582.910 $\pm$ 41.259	0.040 $\pm$ 0.004
9	3697.730 $\pm$ 41.058	0.045 $\pm$ 0.005
10	3728.210 $\pm$ 39.349	0.058 $\pm$ 0.006
20	4006.310 $\pm$ 17.010	0.065 $\pm$ 0.004
30	4061.970 $\pm$ 5.005	0.099 $\pm$ 0.008
40	4070.600 $\pm$ 4.346	0.091 $\pm$ 0.008
50	4069.680 $\pm$ 5.139	0.102 $\pm$ 0.009
60	4074.920 $\pm$ 1.028	0.118 $\pm$ 0.008
70	4074.840 $\pm$ 2.824	0.137 $\pm$ 0.009
80	4076.480 $\pm$ 0.975	0.151 $\pm$ 0.010
90	4075.200 $\pm$ 0.920	0.151 $\pm$ 0.008
100	4074.980 $\pm$ 1.083	0.165 $\pm$ 0.010
200	4075.080 $\pm$ 0.909	0.219 $\pm$ 0.011
300	4074.700 $\pm$ 1.015	0.293 $\pm$ 0.014
400	4074.840 $\pm$ 1.102	0.345 $\pm$ 0.013
448	4075.420 $\pm$ 0.978	0.370 $\pm$ 0.011

Table 3: Performance metrics for different values of *maxneighbour*.

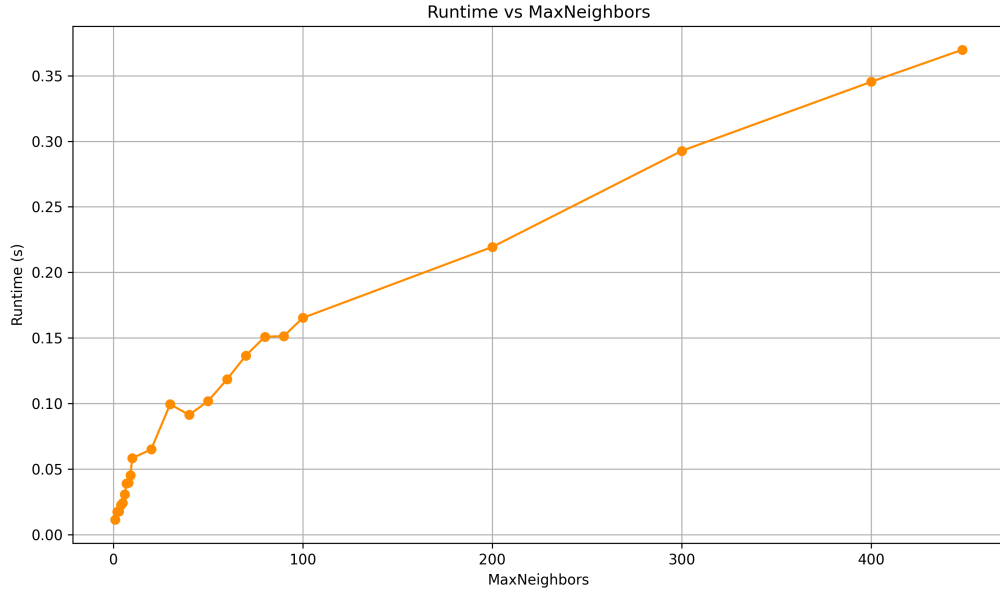


Figure 3: Runtime over different values of *maxneighbour*.

<i>maxneighbour</i>	Micro F1s	Macro F1s	Weighted F1s
1	0.655 $\pm$ 0.010	0.387 $\pm$ 0.014	0.598 $\pm$ 0.011
2	0.672 $\pm$ 0.009	0.399 $\pm$ 0.013	0.615 $\pm$ 0.010
3	0.684 $\pm$ 0.007	0.394 $\pm$ 0.012	0.626 $\pm$ 0.008
4	0.680 $\pm$ 0.008	0.390 $\pm$ 0.014	0.619 $\pm$ 0.010
5	0.680 $\pm$ 0.006	0.385 $\pm$ 0.011	0.620 $\pm$ 0.007
6	0.690 $\pm$ 0.006	0.399 $\pm$ 0.012	0.629 $\pm$ 0.007
7	0.688 $\pm$ 0.006	0.390 $\pm$ 0.012	0.627 $\pm$ 0.007
8	0.686 $\pm$ 0.005	0.384 $\pm$ 0.011	0.626 $\pm$ 0.006
9	0.695 $\pm$ 0.005	0.394 $\pm$ 0.011	0.637 $\pm$ 0.006
10	0.701 $\pm$ 0.005	0.397 $\pm$ 0.010	0.642 $\pm$ 0.006
20	0.694 $\pm$ 0.004	0.379 $\pm$ 0.005	0.634 $\pm$ 0.005
30	0.695 $\pm$ 0.003	0.379 $\pm$ 0.004	0.635 $\pm$ 0.004
40	0.692 $\pm$ 0.005	0.378 $\pm$ 0.004	0.631 $\pm$ 0.005
50	0.690 $\pm$ 0.004	0.379 $\pm$ 0.006	0.629 $\pm$ 0.003
60	0.692 $\pm$ 0.004	0.378 $\pm$ 0.002	0.631 $\pm$ 0.003
70	0.688 $\pm$ 0.002	0.377 $\pm$ 0.001	0.627 $\pm$ 0.002
80	0.688 $\pm$ 0.002	0.378 $\pm$ 0.001	0.627 $\pm$ 0.002
90	0.688 $\pm$ 0.002	0.377 $\pm$ 0.001	0.627 $\pm$ 0.002
100	0.688 $\pm$ 0.001	0.377 $\pm$ 0.001	0.627 $\pm$ 0.001
200	0.688 $\pm$ 0.001	0.377 $\pm$ 0.001	0.626 $\pm$ 0.001
300	0.688 $\pm$ 0.000	0.377 $\pm$ 0.000	0.626 $\pm$ 0.000
400	0.688 $\pm$ 0.000	0.377 $\pm$ 0.000	0.626 $\pm$ 0.000
448	0.688 $\pm$ 0.000	0.377 $\pm$ 0.000	0.626 $\pm$ 0.000

Table 4: F1 scores for different values of *maxneighbour*.

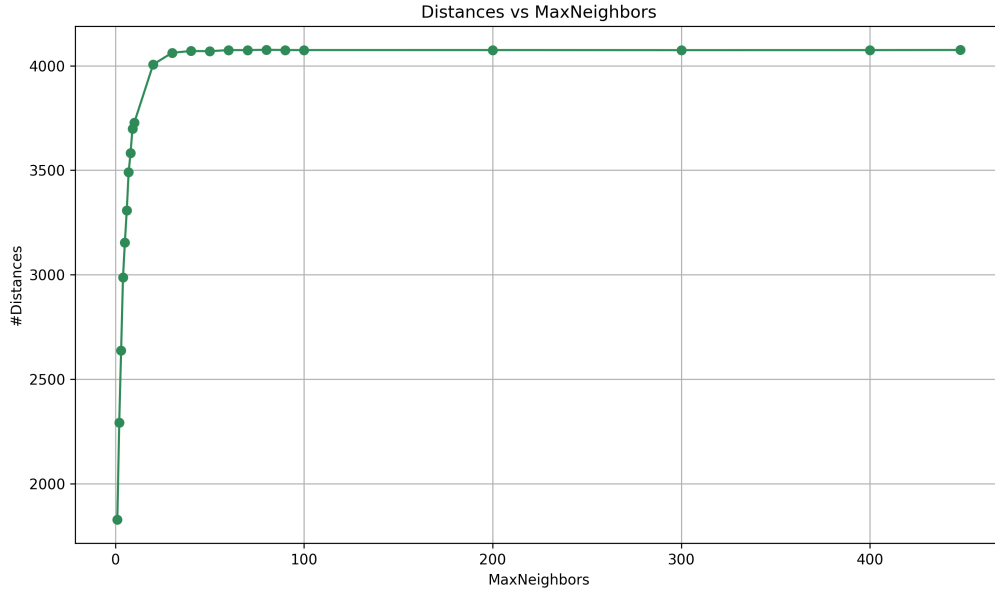


Figure 4: Number of distances computed over different values of *maxneighbour*.

	Micro Recalls	Macro Recalls	Weighted Recalls	Weighted Precisions	Micro Precisions	Macro Precisions
1	0.655 $\pm$ 0.010	0.430 $\pm$ 0.015	0.655 $\pm$ 0.010	0.594 $\pm$ 0.013	0.655 $\pm$ 0.010	0.400 $\pm$ 0.016
2	0.672 $\pm$ 0.009	0.445 $\pm$ 0.014	0.672 $\pm$ 0.009	0.614 $\pm$ 0.014	0.672 $\pm$ 0.009	0.415 $\pm$ 0.017
3	0.684 $\pm$ 0.007	0.438 $\pm$ 0.014	0.684 $\pm$ 0.007	0.616 $\pm$ 0.011	0.684 $\pm$ 0.007	0.405 $\pm$ 0.016
4	0.680 $\pm$ 0.008	0.439 $\pm$ 0.016	0.680 $\pm$ 0.008	0.607 $\pm$ 0.012	0.680 $\pm$ 0.008	0.395 $\pm$ 0.016
5	0.680 $\pm$ 0.006	0.433 $\pm$ 0.013	0.680 $\pm$ 0.006	0.608 $\pm$ 0.011	0.680 $\pm$ 0.006	0.390 $\pm$ 0.015
6	0.690 $\pm$ 0.006	0.445 $\pm$ 0.013	0.690 $\pm$ 0.006	0.613 $\pm$ 0.011	0.690 $\pm$ 0.006	0.401 $\pm$ 0.015
7	0.688 $\pm$ 0.006	0.438 $\pm$ 0.013	0.688 $\pm$ 0.006	0.609 $\pm$ 0.011	0.688 $\pm$ 0.006	0.389 $\pm$ 0.016
8	0.686 $\pm$ 0.005	0.431 $\pm$ 0.012	0.686 $\pm$ 0.005	0.610 $\pm$ 0.010	0.686 $\pm$ 0.005	0.385 $\pm$ 0.015
9	0.695 $\pm$ 0.005	0.443 $\pm$ 0.011	0.695 $\pm$ 0.005	0.619 $\pm$ 0.010	0.695 $\pm$ 0.005	0.391 $\pm$ 0.014
10	0.701 $\pm$ 0.005	0.447 $\pm$ 0.010	0.701 $\pm$ 0.005	0.623 $\pm$ 0.010	0.701 $\pm$ 0.005	0.392 $\pm$ 0.014
20	0.694 $\pm$ 0.004	0.432 $\pm$ 0.005	0.694 $\pm$ 0.004	0.606 $\pm$ 0.006	0.694 $\pm$ 0.004	0.362 $\pm$ 0.007
30	0.695 $\pm$ 0.003	0.431 $\pm$ 0.003	0.695 $\pm$ 0.003	0.609 $\pm$ 0.006	0.695 $\pm$ 0.003	0.362 $\pm$ 0.006
40	0.692 $\pm$ 0.005	0.429 $\pm$ 0.004	0.692 $\pm$ 0.005	0.602 $\pm$ 0.005	0.692 $\pm$ 0.005	0.360 $\pm$ 0.007
50	0.690 $\pm$ 0.004	0.431 $\pm$ 0.006	0.690 $\pm$ 0.004	0.601 $\pm$ 0.004	0.690 $\pm$ 0.004	0.361 $\pm$ 0.007
60	0.692 $\pm$ 0.004	0.430 $\pm$ 0.002	0.692 $\pm$ 0.004	0.603 $\pm$ 0.003	0.692 $\pm$ 0.004	0.360 $\pm$ 0.002
70	0.688 $\pm$ 0.002	0.428 $\pm$ 0.001	0.688 $\pm$ 0.002	0.599 $\pm$ 0.002	0.688 $\pm$ 0.002	0.358 $\pm$ 0.002
80	0.688 $\pm$ 0.002	0.429 $\pm$ 0.001	0.688 $\pm$ 0.002	0.600 $\pm$ 0.001	0.688 $\pm$ 0.002	0.360 $\pm$ 0.001
90	0.688 $\pm$ 0.002	0.429 $\pm$ 0.001	0.688 $\pm$ 0.002	0.600 $\pm$ 0.002	0.688 $\pm$ 0.002	0.360 $\pm$ 0.001
100	0.688 $\pm$ 0.001	0.429 $\pm$ 0.001	0.688 $\pm$ 0.001	0.599 $\pm$ 0.002	0.688 $\pm$ 0.001	0.358 $\pm$ 0.002
200	0.688 $\pm$ 0.001	0.428 $\pm$ 0.001	0.688 $\pm$ 0.001	0.599 $\pm$ 0.000	0.688 $\pm$ 0.001	0.359 $\pm$ 0.000
300	0.688 $\pm$ 0.000	0.428 $\pm$ 0.000	0.688 $\pm$ 0.000	0.599 $\pm$ 0.000	0.688 $\pm$ 0.000	0.360 $\pm$ 0.000
400	0.688 $\pm$ 0.000	0.428 $\pm$ 0.000	0.688 $\pm$ 0.000	0.599 $\pm$ 0.000	0.688 $\pm$ 0.000	0.360 $\pm$ 0.000
448	0.688 $\pm$ 0.000	0.428 $\pm$ 0.000	0.688 $\pm$ 0.000	0.599 $\pm$ 0.000	0.688 $\pm$ 0.000	0.360 $\pm$ 0.000

Table 5: Recall and precision scores for different values of *maxneighbour*.

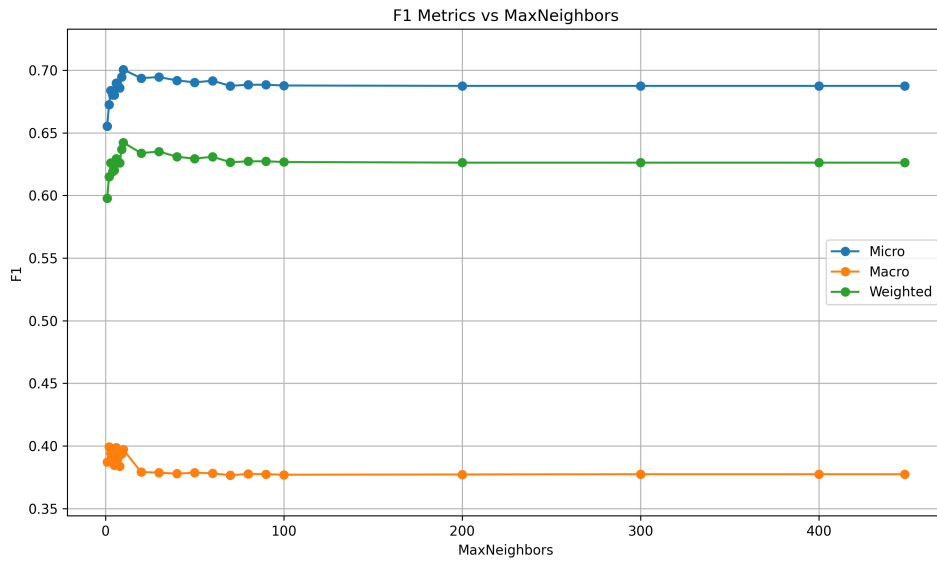


Figure 5: F1 metrics (micro, macro, weighted) vs *maxneighbour*.

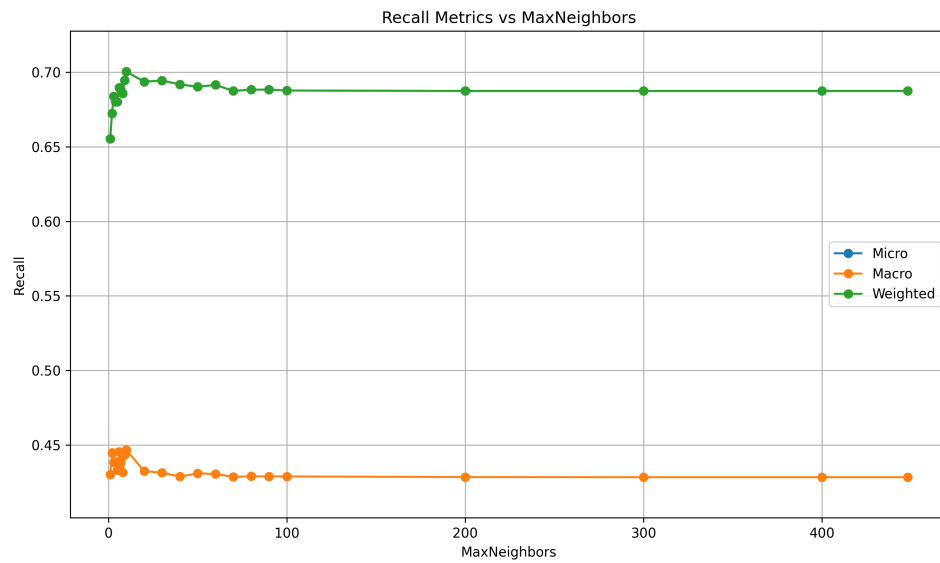


Figure 6: Recall metrics (micro, macro, weighted) vs *maxneighbour*.

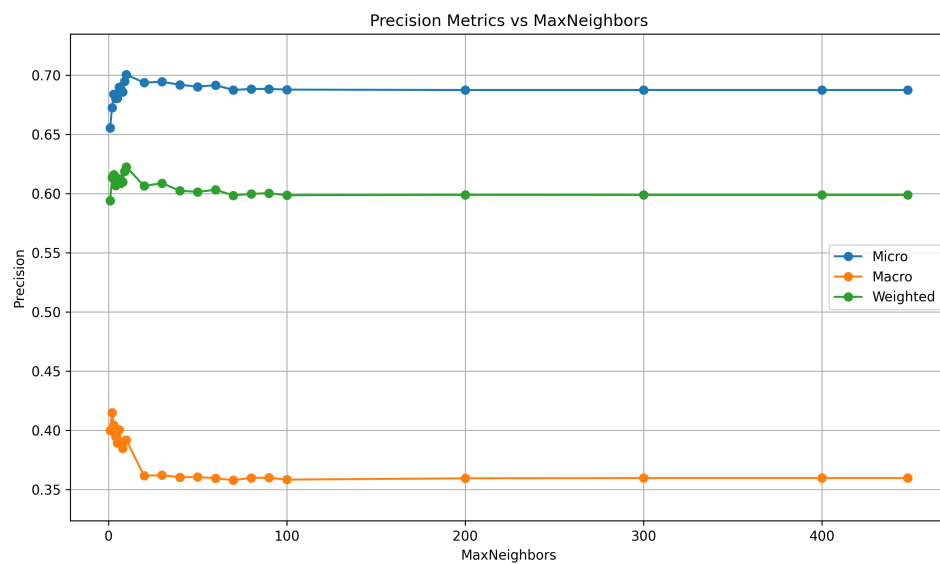


Figure 7: Precision metrics (micro, macro, weighted) vs *maxneighbour*.